

MATERIAL DE APOYO · UNIDAD INICIAL

Algoritmos, pseudocódigo y diagramas de flujo

Guía directa para aprender a plantear una solución antes de programarla: entender el problema, organizar los pasos, representarlos y comprobarlos con PSeInt.

Autor: Rubén Pérez Ibáñez · Programación · 1.º DAW · IES Álvaro Falomir ·
Curso 2026/27

Contenido

1. Qué es un algoritmo

2. Entrada · Proceso · Salida

3. Pseudocódigo y PSeInt

4. Variables y operadores

5. Escribir, Leer y asignar

6. Condicionales

7. Bucles

8. Diagramas de flujo

9. Trazas

10. Método de resolución

01 · LA IDEA PRINCIPAL

¿Qué es un algoritmo?

Un algoritmo es una secuencia ordenada de pasos que permite resolver un problema. En programación, esos pasos deben ser suficientemente claros para que puedan ejecutarse sin ambigüedad.

Preciso

Indica qué debe hacerse y en qué orden.
No debe depender de interpretaciones personales.

Definido

Con los mismos datos de entrada debe seguir la misma lógica y producir el resultado previsto.

Finito

Debe terminar después de un número limitado de pasos. Un ciclo que nunca acaba es un error de diseño.

Además, una buena solución debe...

Ser correcta

Resolver realmente el problema planteado y producir resultados válidos.

Ser legible

Usar nombres claros, una estructura sencilla y mensajes comprensibles.

Ser robusta

Prever datos límite o entradas incorrectas cuando el problema lo requiera.

Objetivo de esta unidad

Aprender a pensar la solución antes de preocuparnos por la sintaxis de Java. Primero trabajamos la lógica; después la trasladaremos a un lenguaje de programación.

02 · MODELO MENTAL

Entrada → Proceso → Salida

Para empezar a resolver un problema conviene separar tres elementos: los datos que necesitamos, las operaciones que realizaremos y el resultado que debe producir el algoritmo.

Entrada

Datos que recibe el algoritmo
Ej.: base y altura



Proceso

Cálculos, decisiones o repeticiones
 $\text{área} \leftarrow \text{base} * \text{altura}$



Salida

Resultado que se muestra o devuelve
Mostrar el área

Ejemplo: área de un rectángulo

Entrada: base y altura. **Proceso:** multiplicarlas. **Salida:** el área calculada.

03 · ESCRIBIR LA SOLUCIÓN

Pseudocódigo y PSeInt

El pseudocódigo expresa un algoritmo mediante instrucciones fáciles de leer, sin depender todavía de un lenguaje de programación concreto. PSeInt utiliza un pseudolenguaje en castellano y permite ejecutar los algoritmos y visualizar su diagrama de flujo.

Estructura mínima

```
Algoritmo HolaMundo
  Escribir "Hola mundo";
FinAlgoritmo
```

Estructura habitual

```
Algoritmo Nombre
  // Declaraciones
  Definir dato Como Entero;

  // Cuerpo del algoritmo
  Leer dato;
  Escribir dato;
FinAlgoritmo
```

En nuestros ejercicios utilizaremos tres ideas básicas: **declarar** los datos que necesitaremos, **leer** los valores de entrada y **escribir** los resultados. Después añadiremos decisiones y bucles.

Forma de trabajar

No intentes memorizar todas las instrucciones. Escribe un ejemplo pequeño, ejecútalo, modifica una línea y observa qué cambia.

04 · TRABAJAR CON INFORMACIÓN

Variables, tipos de datos y operadores

Una variable es un nombre asociado a un dato cuyo valor puede cambiar durante la ejecución. Es recomendable usar nombres que expliquen qué información contienen.

Tipo	Qué guarda	Ejemplos
Entero	Números sin decimales	0, 18, -3
Real	Números con decimales	7.5, 21.25
Cadena	Texto	"Ana", "DAW"
Logico	Verdadero o falso	Verdadero, Falso

```
Definir edad Como Entero;  
Definir precio Como Real;  
Definir nombre Como Cadena;  
Definir encontrado Como Logico;
```

Operadores más utilizados

Grupo	Operadores	Ejemplo
Aritméticos	+ - * / MOD	resto <- numero MOD 2
Relacionales	= <> < > <= >=	edad >= 18
Lógicos	Y O NO	edad >= 18 Y edad < 65
Asignación	<-	resultado <- a + b

No confundir

`resultado <- a + b` significa «calcula la expresión y guarda el valor en resultado». La flecha representa una asignación, no una igualdad matemática.

Escribir, Leer y asignar

1. Mostrar información con Escribir

```
Algoritmo Presentacion
  Escribir "Hola";
  Escribir "Estoy aprendiendo programación";
FinAlgoritmo
```

2. Pedir información con Leer

```
Algoritmo Saludar
  Definir nombre Como Cadena;

  Escribir "¿Cómo te llamas?";
  Leer nombre;

  Escribir "Hola ", nombre;
FinAlgoritmo
```

3. Realizar un cálculo

```
Algoritmo Suma
  Definir a, b, resultado Como Entero;

  Escribir "Introduce el primer número:";
  Leer a;
  Escribir "Introduce el segundo número:";
  Leer b;

  resultado <- a + b;
  Escribir "La suma es ", resultado;
FinAlgoritmo
```

Secuencia recomendada

Escribe → ejecuta → cambia algo → vuelve a ejecutar. Esa comparación permite entender para qué sirve cada instrucción mucho más rápido que memorizar definiciones.

06 · TOMAR DECISIONES

Condicionales: Si y SiNo

Una condición se evalúa como verdadera o falsa. A partir de ese resultado el algoritmo decide qué instrucciones ejecutar.

Condición simple

```
Si nota >= 5 Entonces  
    Escribir "Aprobado";  
FinSi
```

Solo se ejecuta el bloque si la condición es verdadera.

Condición doble

```
Si edad >= 18 Entonces  
    Escribir "Mayor de edad";  
SiNo  
    Escribir "Menor de edad";  
FinSi
```

Se ejecuta una rama u otra, pero no las dos.

Consejo

Antes de escribir el **Si**, expresa la condición como una pregunta de respuesta Sí/No. Por ejemplo: «¿la nota es mayor o igual que 5?».

07 · REPETIR CON CONTROL

Mientras, Repetir y Para

Un bucle repite un bloque de instrucciones. La elección depende de cuándo se comprueba la condición y de si conocemos o no el número de repeticiones.

Mientras

Comprueba la condición **antes**. Puede ejecutarse cero veces.

```
Mientras n > 0 Hacer
  Escribir n;
  n <- n - 1;
FinMientras
```

Repetir

Comprueba al final. El bloque se ejecuta **al menos una vez**.

```
Repetir
  Leer n;
Hasta Que n > 0
```

Para

Es apropiado cuando conocemos cuántas repeticiones necesitamos.

```
Para i <- 1 Hasta 10 Hacer
  Escribir i;
FinPara
```

Error clásico

En un Mientras, alguna instrucción debe modificar los datos que intervienen en la condición. Si nada cambia, podemos crear un bucle infinito.

08 · VER EL ALGORITMO

Diagramas de flujo u ordinogramas

Un diagrama de flujo representa visualmente la secuencia de un algoritmo. Para el programa utilizaremos un diagrama detallado -también llamado **ordinograma**- que muestra inicio, operaciones, entradas/salidas, decisiones y final.



Terminal

Marca el comienzo o el final.



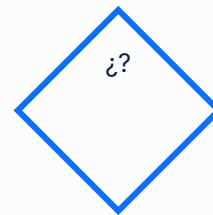
Entrada / salida

Datos que entran o información que mostramos.



Proceso

Cálculos, asignaciones y acciones.



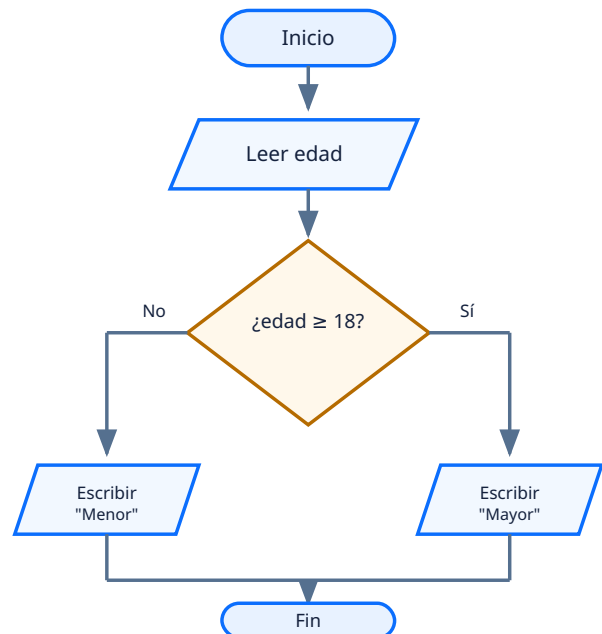
Decisión

Condición que abre dos o más caminos.

Ejemplo: determinar si una persona es mayor de edad

```
Algoritmo MayorEdad
  Definir edad Como Entero;
  Leer edad;

  Si edad >= 18 Entonces
    Escribir "Mayor de edad";
  SiNo
    Escribir "Menor de edad";
  FinSi
FinAlgoritmo
```



Reglas prácticas para dibujarlos

- Debe existir un inicio claro y un final claro.

- Todos los símbolos deben estar conectados mediante líneas de flujo.
- Las flechas indican el orden de ejecución, normalmente de arriba abajo y de izquierda a derecha.
- En una decisión, identifica las salidas: por ejemplo, **Sí** y **No**.
- Evita cruzar líneas; utiliza un recorrido visual limpio.
- El diagrama y el pseudocódigo deben representar exactamente la misma lógica.

09 · COMPROBAR ANTES DE EJECUTAR

Trazas: seguir un algoritmo paso a paso

Una **traza** consiste en simular manualmente la ejecución y anotar cómo cambian las variables. Es especialmente útil para encontrar errores en condicionales y bucles.

Ejemplo

```
Algoritmo Contar
  Definir i Como Entero;
  i <- 1;

  Mientras i <= 3 Hacer
    Escribir i;
    i <- i + 1;
  FinMientras
FinAlgoritmo
```

Tabla de traza

Paso	Valor de i	Salida
Inicialización	1	-
Iteración 1	1 → 2	1
Iteración 2	2 → 3	2
Iteración 3	3 → 4	3
Fin	4	-

Consejo

Si un bucle no hace lo que esperas, recorre dos o tres iteraciones a mano. Anotar los valores de las variables suele mostrar rápidamente dónde está el error.

10 · CÓMO AFRONTAR UN PROBLEMA

Método de trabajo recomendado

- 1 Lee el problema y detecta qué te piden.**
No empieces escribiendo instrucciones todavía.
- 2 Identifica entrada, proceso y salida.**
Anota los datos necesarios y el resultado que debes obtener.
- 3 Describe la solución en lenguaje natural.**
Ej.: leer edad → comprobar si es ≥ 18 → mostrar resultado.
- 4 Pásala a pseudocódigo.**
Utiliza únicamente las estructuras que necesitas.
- 5 Haz una traza y prueba casos distintos.**
Incluye valores normales, límites y casos problemáticos.
- 6 Ejecuta, corrige y simplifica.**
Una solución correcta también debe ser legible.

Continúa practicando

La guía explica los conceptos. La página de ejercicios te obliga a utilizarlos y valida automáticamente los elementos esenciales de cada solución.

[Abrir ejercicios autocorregibles](#)

[Ejecutar en PSeInt Web ↗](#)